# TP1 — Métriques de performance et lois fondamentales

> **Cours :** Performance et Optimisation des Systèmes — L3 DAR, Semaine 1
> **Auteur :** El Hadji Fallou Bousso
> **Université :** Université Cheikh Akhmadoul Khadim
> **Dépôt Git :** github.com/serignefalloubousso99-dot/tp-perf-l3

## Environnement de mesure

J'ai testé le service `service-commandes` (Node.js, port 8080) sous WSL2 (Ubuntu 24.04), sur mon PC Windows. Le générateur de charge ( `ab` ) et le service tournent sur la même machine : `ab` consomme donc un peu de CPU pendant les tests, ce qui peut légèrement baisser les résultats.

---

## Partie A — Métriques de base

### Exercice 2.1 — Latence d'une requête isolée

Commande : `curl` sur `/commande?montant=100&iterations=200000`, répétée 10 fois.

| # | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| temps (s) | 0.0084 | 0.0066 | 0.0033 | 0.0046 | 0.0041 | 0.0034 | 0.0036 | 0.0046 | 0.0035 | 0.0034 |

| moyenne | médiane | min | max |
|---|---|---|---|
| 4.6 ms | 3.9 ms | 3.3 ms | 8.4 ms |

**Latence ou débit ?** C'est une mesure de **latence** : une seule requête à la fois, pas de notion de débit ici.

La première requête est plus lente (8.4 ms) : le service met un petit moment à monter en régime.

### Exercice 2.2 — Débit sous charge

Commande `ab` avec différentes concurrences :

| Concurrence | Débit (req/s) | Temps par requête (client) | Temps par requête (moyen serveur) |
|---|---|---|---|
| 10 | 482 | 20.7 ms | 2.1 ms |
| 50 | 534 | 93.7 ms | 1.9 ms |
| 100 | 557 | 179.5 ms | 1.8 ms |

**Pourquoi les deux temps diffèrent ?** Le premier est le temps vu par un seul client (il attend son tour). Le second est le temps moyen "machine" (1000 / débit) — logique qu'il soit plus petit.

**Le débit augmente-t-il indéfiniment ?** Non : il passe de 482 à 557 req/s entre `c=10` et `c=100`, une hausse de plus en plus faible. Le service ne traite qu'une requête à la fois (il est mono-thread), donc ajouter des clients ne fait qu'allonger la file d'attente, sans augmenter la vitesse de traitement.

*Remarque :* `ab` signale quelques "échecs" (2 à 96 selon le test), mais ce n'est pas une vraie panne : la taille de la réponse JSON change légèrement d'une requête à l'autre (le champ `duree_ms` n'a pas toujours le même nombre de chiffres), et `ab` compare les tailles entre elles.

### Exercice 2.3 — Percentiles

Commande `ab -n 500 -c 20` avec export CSV.

| Moyenne | p50 | p95 | p99 | max |
|---|---|---|---|---|
| 39.7 ms | 37.7 ms | 59.8 ms | 67.4 ms | 68.3 ms |

**Écart moyenne / p95 ?** Le p95 est 50 % plus élevé que la moyenne : la plupart des requêtes sont rapides, mais une petite partie (5 %) est nettement plus lente.

**SLA sur la moyenne ou le p95 ?** Sur le **p95**. Un engagement de service doit protéger la majorité des utilisateurs, pas seulement le cas moyen.

---

## Partie B — Loi d'Amdahl

### Exercice 3.1 — Calculs

Traitement : 3 min (séquentiel) + 42 min (parallélisable) + 5 min (séquentiel) = 50 min.

- Fraction parallélisable : `p = 42/50 = 0.84`
- Speedup maximal : `S_max = 1/(1-p) = 6.25`

| N | S(N) |
|---|---|
| 4 | 2.70 |
| 16 | 4.71 |
| 64 | 5.78 |

En comparant `S(2N)` à `S(N)`, le gain marginal passe sous 5 % entre **N = 64 et N = 128** (+3.9 %). Au-delà, ajouter des processeurs ne sert presque plus à rien.

**128 processeurs au lieu de 32 ?** À N=32, le speedup est déjà de 5.37 (86 % du maximum théorique). Passer à 128 (4x plus de matériel) ne fait gagner que 11 % de plus. Ce n'est **pas rentable** : mieux vaut garder 32 processeurs et essayer de réduire la partie séquentielle (16 %).

### Exercice 3.2 — Vérification (bonus)

Test avec 1, 2 et 4 workers Node.js (`WORKERS=N`) :

| Workers | Débit (req/s) | Speedup |
|---|---|---|
| 1 | 442 | 1.00 |
| 2 | 874 | 1.98 |
| 4 | 1418 | 3.20 |

En résolvant `S(4) = 1/((1-p)+p/4)`, on trouve **p ≈ 0.92** (92 % parallélisable), proche de 100 % mais pas tout à fait.

**Pourquoi pas 100 % ?** Tous les workers écrivent dans le même fichier `commandes.log`. Ces écritures ne peuvent pas se faire en même temps, ce qui freine un peu le gain de parallélisme.

## Partie C — Loi de Little

### Exercice 4.1 — Dimensionnement (données de l'exercice 2.3)

```
λ = 503.9 req/s
W = 0.0397 s
L = λ × W ≈ 20
```

Il y a donc en moyenne **20 requêtes en attente** en même temps dans le service. Or le service ne traite qu'une seule requête à la fois (pas de vrai pool de threads dans ce code) : ce résultat confirme qu'il n'est pas dimensionné pour gérer 20 clients en parallèle. Passer le service en plusieurs workers (comme en 3.2) est la solution.

### Exercice 4.2 — Cas d'étude chiffré

Service de notification : λ = 1200/s.

- W = 150 ms → `L = 1200 × 0.150 = 180` notifications en cours.
- Nouveau fournisseur, W = 400 ms → `L = 1200 × 0.400 = 480` notifications en cours.
- Avec 500 connecteurs par serveur : `480/500 = 96 %` d'occupation, donc un seul serveur suffit *à peine*. Par sécurité, il faut prévoir **2 serveurs** après le changement de fournisseur.

## Conclusion

Les lois théoriques (Amdahl, Little) donnent une bonne estimation, mais la réalité ajoute toujours un peu de friction (fichier partagé, service mono-thread) qui n'apparaît pas dans les formules. La loi de Little a surtout montré que le service n'a pas de vrai système pour traiter plusieurs requêtes en même temps. Et la moyenne seule cache souvent les cas lents : le p95 est plus honnête.